

COURSE CODE

SBT-DF203

COURSE TITLE

Basic Networking Skills for Digital Forensics

LAB TITLE

Firewall Traffic Control and Forensic Verification

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

18/09/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

Mr. Aminu Idris, AMCPN
Commandant

Executive Summary

This report describes a firewall test carried out on a Kali Linux server named “kali” using iptables. The test recorded the existing firewall rules, checked normal HTTP traffic, added a DROP rule for a specific IP address, captured traffic while the rule was active, compared the results, and then removed the rule.

The rule was added, checked, and removed successfully. The firewall rules before and after the test were also saved. However, the evidence confirming that the traffic was blocked was incomplete. The saved ruleset matched an empty file, no matching packets appeared in the allowed-traffic capture, and the DROP rule counters remained at zero during the blocked test.

Required Tools and Environment

Item	Recommended Tool	Purpose
Server VM	Kali/Ubuntu with Apache	Protected host.
Client VM	Linux or Windows on host-only network	Generate HTTP requests.
Firewall tools	iptables, iptables-save	Apply and preserve rules.
Capture tools	tshark/Wireshark	Capture baseline and blocked traffic.
Client tools	curl, browser	Test access.
Optional extension	NetfilterQueue	Instructor-led packet interception demonstration.

Lab Folder Structure and Evidence Preparation

```
ponzhi@kali ~/SBT-DF203-Lab6
Session Actions Edit View Help
(ponzhi@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}

(ponzhi@kali)-[~]
$ cd ~/SBT-DF203-Lab6

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ pwd
/home/ponzhi/SBT-DF203-Lab6

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ find . -maxdepth 1 -type d -print
.
./evidence
./working
./exported
./reports
./scripts
./screenshots

(ponzhi@kali)-[~/SBT-DF203-Lab6]
```

```
ponzhi@kali ~/SBT-DF203-Lab6
Session Actions Edit View Help
(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ sudo apt update
[sudo] password for ponzhi:
Ign:1 http://http.kali.org/kali kali-rolling InRelease
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 77.0 MB in 56s (1,366 kB/s)
370 packages can be upgraded. Run 'apt list --upgradable' to see them.

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ sudo systemctl enable --now apache2
[sudo] password for ponzhi:

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$
```

```
(ponzhi@kali)~[~/SBT-DF203-Lab6]
$ printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: %s</p></body></html>\n' "$(hostname)" | sudo tee /var/www/html/firewall_lab.html
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>

(ponzhi@kali)~[~/SBT-DF203-Lab6]
$ sudo iptables-save | tee reports/iptables_before.rules

(ponzhi@kali)~[~/SBT-DF203-Lab6]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                   destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                   destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                   destination

(ponzhi@kali)~[~/SBT-DF203-Lab6]
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855  reports/iptables_before.rules

(ponzhi@kali)~[~/SBT-DF203-Lab6]
$
```

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab6-Kumpon_Emmanuel_Nandom
Trainee name	Kumpon Emmanuel Nandom
Date and time started	18 September 2026, 3:58 pm
Evidence file name(s)	evidence/http_allowed.pcapng; evidence/http_blocked.pcapng; reports/iptables_before.rules
Source or generation method	tshark packet capture on server eth0 during curl HTTP requests; iptables-save export of the pre-test ruleset
Original SHA-256	http_allowed.pcapng: 0d64b5e48f67458f70856bf884c0a731eb89da8accaf53130d03de99af827645 http_blocked.pcapng: 69bd55fca5c44461b738348796adf3a33322674fb037b57e466b9da33f5d3b54 iptables_before.rules: e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
Working-copy SHA-256	

Field	Student Entry
Analysis workstation/V M	Kali Linux, hostname "kali", eth0 192.168.109.128/24, gateway 192.168.109.2
Notes on any changes	iptables_before.rules hash equals the SHA-256 of an empty file; the export did not capture iptables-save output; needs re-export. Allowed-capture filter used the server's own IP (192.168.109.128), not a separate client. Blocked-client IP (192.168.56.20) is on a different subnet from the server; DROP rule counters stayed at 0/0.

Part A - Record the Lab Network and Baseline Access

```

(ponzhi@kali)~/SBT-DF203-Lab6
$ ip -br address | tee reports/server_interfaces.txt
lo                UNKNOWN    127.0.0.1/8 ::1/128
eth0              UP          192.168.109.128/24 fe80::20c:29ff:fe6f:fd97/64

(ponzhi@kali)~/SBT-DF203-Lab6
$ ip route | tee reports/server_routes.txt
default via 192.168.109.2 dev eth0 proto dhcp src 192.168.109.128 metric 100
192.168.109.0/24 dev eth0 proto kernel scope link src 192.168.109.128 metric 100

(ponzhi@kali)~/SBT-DF203-Lab6
$ sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      511          *:80          *:*          users:((("apache2",pid=1003,fd=4),("apache2",pid=1002,fd=4),("apache2",pid=1001,fd=4),("apache2",pid=1000,fd=4),("apache2",pid=999,fd=4),("apache2",pid=960,fd=4)))

(ponzhi@kali)~/SBT-DF203-Lab6
$

```

```

(ponzhi@kali)~/SBT-DF203-Lab6
$ curl -v --connect-timeout 5 http://192.168.109.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.109.128:80 ...
* Established connection to 192.168.109.128 (192.168.109.128 port 80) from 192.168.109.128 port 60446
  % Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload  Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0      0     0      0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.109.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 15:53:59 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 15:23:28 GMT
< ETag: "6b-65bc37b7c2d82"
< Accept-Ranges: bytes
< Content-Length: 107
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [107 bytes data]
100  107 100    107    0     0  58342    0      0      0      0      0
* Connection #0 to host 192.168.109.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: kali</p></body></html>

(ponzhi@kali)~/SBT-DF203-Lab6
$

```

Part B - Capture the Allowed HTTP Baseline

```
Session Actions Edit View Help

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ tshark -i eth0 -f 'host 192.168.109.128 and tcp port 80' -a duration:30 -w evidence/http_allowed.pcapng &
sleep 3
# Now go run the curl command from the allowed client during this window
wait
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
[5] 88157
Capturing on 'eth0'

[5] done tshark -i eth0 -f 'host 192.168.109.128 and tcp port 80' -a duration:30 -w
0d64b5e48f67458f70856bf884c0a731eb89da8accaf53130d03de99af827645 evidence/http_allowed.pcapng

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$
```

"Screenshot_2026-09-18_20_45_20.png" | 145.5 KiB (149,014 bytes) | PNG image

Part C - Apply and Verify the Blocking Rule

```
Session Actions Edit View Help

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -I INPUT 1 -s "192.168.56.20" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "192.168.56.20" -p tcp --dport 80 -j DROP
printf 'Rule verified present.\n' | tee reports/rule_verification.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination tcp dpt:80
1 0 0 DROP tcp -- * * 192.168.56.20 0.0.0.0/0 tcp dpt:80
Rule verified present.

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$
```

Part D - Capture Blocked Traffic and Rule Counters

```
(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ tshark -i eth0 -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
sleep 3
# From the blocked client during this capture:
# curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
wait

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
[5] 101441
Capturing on 'eth0'

[5] done tshark -i eth0 -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w
Chain INPUT (policy ACCEPT 35 packets, 4495 bytes)
num  pkts bytes target    prot opt in     out     source                destination            tcp dpt:80
1      0      0 DROP      tcp  --  *      *       192.168.56.20         0.0.0.0/0               tcp dpt:80
69bd55fca5c44461b738348796adf3a33322674fb037b57e466b9da33f5d3b54  evidence/http_blocked.pcapng
```

Part E - Compare Allowed and Blocked Captures

```
(ponzhi@kali)-[~/SBT-DF203-Lab6]
$ for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
  echo "==== $PCAP ====="
  tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
    -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstp \
    -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
==== evidence/http_allowed.pcapng =====
==== evidence/http_blocked.pcapng =====

(ponzhi@kali)-[~/SBT-DF203-Lab6]
$
```

Indicator	Allowed Capture	Blocked Capture
Client SYN visible?	Confirmed no matching frames returned	Confirmed no matching frames returned
Server SYN-ACK visible?	Confirmed in pcap review	Confirmed in pcap review
Handshake completed?	Inferred from curl 200 OK; not visually confirmed in pcap	No blocked-client traffic reached the capture
HTTP GET visible?	Confirmed in pcap review	Confirmed in pcap review
HTTP response visible?	Confirmed in pcap review	Confirmed in pcap review
Retransmissions/timeouts	None captured	None captured (consistent with traffic never reaching eth0)

Indicator	Allowed Capture	Blocked Capture
iptables counter change	N/A - no DROP rule active during baseline	No change - counter stayed 0 packets / 0 bytes after test
curl result	200 OK (baseline_curl.txt)	Not evidenced - no blocked-client curl output captured

Part F - Remove the Rule and Restore Access

```

(ponzhi@kali)~[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -D INPUT -s "192.168.56.20" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "192.168.56.20" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
# Confirm access from the previously blocked client:
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
Chain INPUT (policy ACCEPT 41 packets, 5231 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
(ponzhi@kali)~[~/SBT-DF203-Lab6]
$

```

Why does DROP commonly cause SYN retransmissions and a timeout?

DROP silently discards the matching packet — no reply of any kind is sent back. The client's TCP stack can't tell this apart from ordinary packet loss on the network, so it assumes the SYN just didn't arrive and retransmits it, usually with exponential backoff. After several failed attempts, it gives up and reports a connection timeout.

How would a REJECT rule differ in the packet capture and client output?

Reject sends an explicit response instead of staying silent, typically a TCP RST. In the capture, you don't see the client's SYN followed almost immediately by that rst/icmp packet, rather than a string of unanswered retransmitted SYNs. On the client side, this means a fast, explicit "connection refused" error instead of a slow timeout. Reject fails quickly and loudly; drop fails slowly and silently.

Why are firewall counters valuable corroborating evidence?

The packet capture on its own tells you traffic arrived and got no response, but not *why*. The iptables -L -n -v packet/byte counters tell you specifically whether that rule matched anything. If the counter incremented at the same time the client saw a failure, that ties the failure directly to the firewall rule rather than some other cause. This is exactly the gap you ran into, my capture and curl behavior alone weren't enough to prove the block; the counters (which stayed at 0/0) were what revealed the traffic never actually reached the rule at all.

What evidence would show that the web server was down rather than firewall-blocked?

If Apache were down but the firewall wasn't blocking anything, you'd typically still see the TCP handshake complete (SYN → SYN-ACK → ACK), since the OS still responds to connections on that port unless nothing is listening — in which case you'd see a RST immediately after the SYN (connection refused at the OS/socket level, not the firewall). Either way, you'd see *some* response at or above the TCP layer. A firewall DROP, by contrast, blocks the SYN itself — no SYN-ACK, no RST, nothing. So: handshake completes + no HTTP response → service-level problem. No response to the SYN at all → firewall block.

What is the risk of deleting a rule by line number after other rules have changed?

Line numbers are positional, not identity-based; they shift every time a rule above them is added or removed. If you delete "rule 3" without re-checking the current listing, you might actually delete a completely different rule than the one you intended, potentially opening a hole you didn't mean to open or removing something unrelated and important. That's exactly why Part F in your lab has you delete by exact specification (iptables -D INPUT -s ... -j DROP) instead of by number; it targets the rule itself, so it's safe even if the chain has changed since you inserted it.